

CareQueue — FINAL Antigravity Prompt Pack

How to use:

- Session 1: paste **PHASE 0 + PHASE 1** together.
- Each later phase: start a new Antigravity session and paste **PHASE 0 + that phase** (Phase 0 is short enough to always include; it is the single source of truth).
- Build order for the hackathon: 0+1 → 2 → rehearse demo → 4 → 3 → 5 (only if time).

PHASE 0 — MASTER BRIEF

```
# CAREQUEUE – MASTER BRIEF (source of truth; read fully before any work)

### Product
CareQueue is a responsive web app (PWA-ready) for smart hospital OPD queue
and appointment management. Patients join a doctor's queue remotely, get a
digital token, track live position and estimated wait, and check in via QR
when they arrive. Doctors run their own queue. Reception coordinates
patients and doctors, adds walk-ins, handles priorities and delays.

Problem solved: patients wait hours without knowing their position;
hospitals juggle appointments, walk-ins and emergencies with paper tokens.
CareQueue makes the queue digital, visible, fair and predictable.

### Tech stack (do not deviate)
- React 18 + Vite + JavaScript (no TypeScript)
- Tailwind CSS · React Router v6 · Lucide React icons
- State: ONE React Context + useReducer store ("QueueStore"). No Redux.
- Data: mock data in src/data/*.js. NO backend, NO real auth, NO real SMS.
- Folder structure:
  src/
    app/                router.jsx, providers.jsx, RoleGuard.jsx
    components/ui/       Button, Input, Select, SearchableSelect, Badge, Card,
                        Modal, Table, Toast, StatCard, Sidebar, Navbar,
                        BottomTabBar, EmptyState, ProgressBar
    components/queue/    TokenCard, QueueTable, QueueStatusBadge,
                        PriorityBadge, DoctorStatusBadge, CurrentPatientCard,
                        DoctorQueueCard, ReceptionDoctorCard, NotificationItem,
                        QueueStatsCard
    features/patient/    features/doctor/  features/reception/  features/auth/
    store/              QueueStore.jsx, queueReducer.js, actions.js,
                        selectors.js, notificationsEffect.js
    data/               hospitals.js, departments.js, doctors.js, patients.js,
                        queue.js, appointments.js, users.js
    lib/               waitTime.js, tokens.js, format.js, validators.js
    docs/              STATUS.md, DESIGN.md, DEMO_SCRIPT.md

### Roles (exactly three)
PATIENT · DOCTOR · RECEPTION
Routes are namespaced: /patient/* /doctor/* /reception/*
Login detects role → redirect to /<role>/dashboard. RoleGuard blocks
cross-role access (redirects to the user's own dashboard).
```

```

## Shared vocabulary – use these exact identifiers everywhere
Queue entry status: WAITING | CALLED | IN_CONSULTATION | COMPLETED |
                    SKIPPED | CANCELLED

Queue entry fields:
  id, tokenNumber, patientId, patientName, doctorId, departmentId,
  hospitalId, status, checkedIn (boolean),
  priority (NORMAL | PRIORITY | URGENT | EMERGENCY),
  source (APPOINTMENT | WALK_IN), appointmentTime (ISO or null),
  reasonForVisit (short optional text),
  joinedAt, checkedInAt, calledAt, startedAt, completedAt, skippedAt
Doctor status: AVAILABLE | ON_BREAK | DELAYED | UNAVAILABLE
               (DELAYED carries delayMinutes; queuePaused boolean is separate)
Token scheme: <DepartmentLetter>-<2-digit sequence>, per doctor per day.
  General Medicine=A Cardiology=C Orthopedics=O Pediatrics=P
  Dermatology=D ENT=E      e.g. A-24, C-14, O-09

## State machine – legal transitions and who may trigger them
FROM          ACTION          TO          ACTOR
(none)         JOIN_QUEUE      WAITING      PATIENT
(none)         ADD_WALK_IN     WAITING(+checkedIn) RECEPTION
WAITING        CHECK_IN       WAITING(checkedIn=true) PATIENT(QR) / RECEPTION
WAITING        CALL_NEXT      CALLED       DOCTOR
CALLED         START_CONSULTATION IN_CONSULTATION DOCTOR
IN_CONSULTATION COMPLETE      COMPLETED    DOCTOR
WAITING/CALLED SKIP           SKIPPED       DOCTOR / RECEPTION / system(grace)
IN_CONSULTATION SKIP          SKIPPED       DOCTOR
SKIPPED        REJOIN         WAITING      PATIENT / RECEPTION
WAITING        CANCEL         CANCELLED    PATIENT / RECEPTION
any active     SET_PRIORITY    (same)       RECEPTION only
any active     TRANSFER        WAITING (new token, target doctor) RECEPTION / DOCTOR

Any other transition must be rejected by the reducer (console.warn + no-op).
"Active" = WAITING, CALLED, IN_CONSULTATION.

## Queue rules (core logic – implement exactly)
1. Ordering of a doctor's WAITING entries:
   priority DESC (EMERGENCY > URGENT > PRIORITY > NORMAL)
   → checkedIn DESC
   → APPOINTMENT with appointmentTime <= now before WALK_IN
   → joinedAt ASC
2. CALL_NEXT picks the first WAITING entry with checkedIn === true.
   Never call a patient who has not checked in. If none is checked in,
   the button shows "No checked-in patients".
3. A not-checked-in patient with 0 checked-in patients ahead gets a
   "Please check in" notification; if still not checked in after a
   10-minute grace window they are auto-SKIPPED with a Rejoin option.
4. Only RECEPTION can change priority. Patients can never self-declare.
5. Wait-time v1 (lib/waitTime.js):
   estimatedWaitMin = patientsAhead × doctor.avgConsultationMin
                     + doctor.delayMinutes
   Show as "~35 min". (Smart range estimate is Phase 4.)
6. patientsAhead = number of WAITING + CALLED entries ordered before me
   for the same doctor.
7. A paused queue (queuePaused=true) disables CALL_NEXT and shows
   "Queue paused" to patients; wait times still display.

## Notifications (in-app toast + per-user notification center)
5 ahead      "Your turn is approaching. You are 5 patients away."

```

```

2 ahead      "Please return to the waiting area. ~10 minutes to go."
CALLED       "It's your turn! Please proceed to Room {room}."
SKIPPED      "You missed your turn." + [Rejoin Queue]
Doctor DELAYED "Dr. {name} is running ~{n} min late. New estimate: ~{m} min."
Not checked in & ≤2 ahead "Please check in at reception or scan the hospital QR."
Queue paused  "Dr. {name} has paused the queue briefly. We'll notify you when it resumes."
Thresholds fire once per entry (track sentNotifications on the entry).

```

```

## Design system (write to docs/DESIGN.md in Phase 1; reuse after)
- Background white; section gray #F8FAFC; text navy #0F172A; muted #64748B
- Primary blue #2563EB; accent teal #0D9488
- Status colors: green #16A34A completed/success · amber #D97706 waiting/
  warning/delayed · red #DC2626 emergency/skipped · blue #2563EB called/
  in-consultation · gray cancelled
- Font Inter. Body ≥16px. Tap targets ≥44px. WCAG AA contrast.
- rounded-xl cards, 1px #E2E8F0 borders, shadow-sm. No gradients, no
  glassmorphism, no heavy animation (only 150ms transitions).
- Every screen has ONE obvious primary action (large button).
- Patient: mobile-first with bottom tab bar. Reception: desktop sidebar,
  collapsible on mobile. Doctor: works equally on desktop/tablet/mobile.

```

```

## Scope boundaries – NEVER build or scaffold
Telemedicine/video · medicine delivery · symptom checker or any diagnosis ·
EMR/medical records · payments/billing · inventory · payroll · pharmacy or
lab queues · family/caregiver accounts · in-hospital navigation ·
document uploads · ratings/reviews · multilingual UI (only a language
dropdown field at registration) · voice navigation.

```

```

## Working rules for every session
1. Read docs/STATUS.md and docs/DESIGN.md first (if they exist).
2. Write a short implementation plan before coding; then implement it.
3. Reuse existing components; never duplicate a component that exists.
4. Run `npm run build` and fix ALL errors and warnings before finishing.
5. Update docs/STATUS.md at the end: routes, components, store actions,
  data files, known gaps, how to test.
6. Finish with a summary: routes added, components created, files changed,
  and a 60-second "how to demo" script.
7. Do not add features outside the current phase, even if they seem easy.

```

PHASE 1 — FOUNDATION + PATIENT PORTAL

```

# PHASE 1 – FOUNDATION + PATIENT PORTAL
Follow the MASTER BRIEF above. This is a brand-new project.

```

```

## 1. Project setup
- Scaffold Vite + React + Tailwind + React Router + Lucide with the exact
  folder structure from the brief. Configure Inter via Google Fonts link.
- Create docs/DESIGN.md (copy the design system section verbatim).
- Create docs/STATUS.md (fill at the end of this phase).

```

```

## 2. Mock data (src/data/)
hospitals.js – 3 hospitals: City Care Hospital (Delhi), Apollo Care
  Hospital (Mumbai), Medicare Clinic (Pune). Fields: id, name, city,
  address, departmentIds[].

```

departments.js – General Medicine (A), Cardiology (C), Orthopedics (O), Pediatrics (P), Dermatology (D), ENT (E). Fields: id, name, tokenLetter.

doctors.js – 8 doctors. Fields: id, name, specialization, departmentId, hospitalId, room, avgConsultationMin, status, delayMinutes, queuePaused, todayHours {start:"09:00", end:"13:00"}, photoInitials.

Must include (all at City Care Hospital):

- Dr. Ankit Sharma – General Medicine – Room 204 – avg 9 min – AVAILABLE
- Dr. Priya Verma – Cardiology – Room 310 – avg 12 min – AVAILABLE
- Dr. Rahul Mehta – Orthopedics – Room 118 – avg 10 min – DELAYED 15

patients.js – 14 patients, Indian names (Priya Mehta, Mohit Sharma, Ansh, Rahul Verma, Sneha Iyer, Arjun Nair, ...). Fields: id, name, mobile, city, dob, gender, preferredLanguage.

queue.js – ~16 entries for today across the three City Care doctors.

Dr. Sharma's queue must contain: A-22 Priya Mehta IN_CONSULTATION, A-23 Mohit Sharma WAITING checkedIn, A-24 Ansh WAITING checkedIn, A-25 Rahul Verma WAITING not checkedIn, plus 3 more; one URGENT and one EMERGENCY somewhere in the hospital; 2 COMPLETED; 1 SKIPPED.

appointments.js – 8 appointments today with 15-min slots.

users.js – demo accounts (password "demo" for all):

- patient@demo.com → PATIENT, linked to patient "Ansh" (holds token A-24)
- doctor@demo.com → DOCTOR, linked to Dr. Ankit Sharma
- reception@demo.com → RECEPTION, City Care Hospital

3. QueueStore (src/store/)

State: { currentUser, doctors, queueEntries, appointments, notifications, demoMode }

Actions (implement ALL now, even ones only used by later phases):

- LOGIN {email,password} · LOGOUT · SWITCH_ROLE_DEV {email}
- JOIN_QUEUE {patientId, doctorId, source, reasonForVisit, appointmentTime}
- CANCEL_ENTRY {entryId} · CHECK_IN {entryId} · REJOIN {entryId}
- CALL_NEXT {doctorId} · START_CONSULTATION {entryId}
- COMPLETE {entryId} · SKIP {entryId}
- SET_PRIORITY {entryId, priority} · TRANSFER {entryId, toDoctorId}
- ADD_WALK_IN {patientName, mobile, doctorId, reasonForVisit, priority}
- SET_DOCTOR_STATUS {doctorId, status, delayMinutes}
- PAUSE_QUEUE {doctorId} · RESUME_QUEUE {doctorId}
- PUSH_NOTIFICATION {userId, type, message} · MARK_NOTIFICATION_READ {id}
- BOOK_APPOINTMENT / RESCHEDULE_APPOINTMENT / CANCEL_APPOINTMENT
- SET_DEMO_MODE {on}

The reducer enforces the state-machine table from the brief.

selectors.js: getQueueForDoctor(doctorId) (brief ordering), getPatientsAhead (entryId), getEstimatedWait(entryId), getCurrentPatient(doctorId), getDoctorStats(doctorId), getActiveEntryForPatient(patientId), getHospitalQueue(hospitalId), getUnreadCount(userId).

lib/tokens.js: nextToken(doctorId, date) → "A-26".

notificationsEffect.js: after every mutation, evaluate thresholds for every active entry and PUSH_NOTIFICATION once per threshold.

4. Auth stub + role routing

/login – email + password, demo-account hint box (three accounts listed with a "Use" button each), link to /register (placeholder page for now: "Registration coming soon").

Role redirect on login. RoleGuard on all role routes. / → /login.

DEV-ONLY floating pill (bottom-right, import.meta.env.DEV only):

- "Role: Patient ▼" switcher between the three demo users, and a "Demo mode" toggle.

```

## 5. Patient Portal (mobile-first; bottom tabs Home · Search · Token ·
    Visits · Profile)
/patient/dashboard – greeting; Active Token card if the patient has an
    active entry (token, doctor, ahead, wait, tap → token screen); next
    appointment card; quick actions (Find a doctor, My visits,
    Notifications with unread badge).
/patient/search – search bar (hospital, doctor, department); result cards.
/patient/hospital/:id – hospital header, departments grid, doctors list
    (status badge, waiting count, "~35 min").
/patient/doctor/:id – doctor card: status badge (DELAYED → amber banner
    "Running ~15 min late"), today's hours, room, "Now serving A-22",
    "6 waiting", "Estimated wait ~35 min", optional "Reason for visit"
    input, [Join Queue] primary button. Disabled with reason if UNAVAILABLE
    or patient already has an active entry.
/patient/token/:entryId – FLAGSHIP SCREEN:
    huge token number; doctor, department, room; three stat tiles: Now
    Serving / Patients Ahead / Estimated Wait; ProgressBar of position;
    checked-in badge ("Checked in" green / "Not checked in" amber);
    [I've arrived – Scan QR] opens modal with a placeholder QR image and a
    "Simulate scan" button that dispatches CHECK_IN; [Leave Queue] with
    confirm modal. Status-driven states: CALLED → full-width blue banner
    "It's your turn! Room 204"; IN_CONSULTATION → "In consultation";
    SKIPPED → red state with [Rejoin Queue]; COMPLETED → "Visit complete".
    All values live-update from the store.
/patient/visits – Upcoming (appointments) and Past (COMPLETED entries).
/patient/notifications – list (NotificationItem), mark read on open.
/patient/profile – basic details, logout.

```

6. Demo Mode

When demoMode is on, a store interval advances Dr. Ankit Sharma's queue every 8 s: CALL_NEXT → (3 s) START_CONSULTATION → (5 s) COMPLETE, looping, so the patient's token screen visibly moves during a pitch. Stops when no checked-in patients remain.

7. Deliver

Build passes with no warnings. docs/STATUS.md written. Summary + 60-second demo script (login as patient → open token screen → toggle demo mode).

PHASE 2 — DOCTOR PORTAL + RECEPTION PORTAL

```

# PHASE 2 – DOCTOR + RECEPTION PORTALS
Read docs/STATUS.md and docs/DESIGN.md first. Do NOT rewrite the Patient
Portal or the QueueStore internals; extend them. All three portals read
from and dispatch to the same QueueStore.

## DOCTOR PORTAL /doctor/* – optimize for speed and simplicity
Nav (top bar on mobile, sidebar on desktop): Dashboard · Live Queue ·
Today's Patients · Profile

/doctor/dashboard
- Header: name, specialization, hospital, room; DoctorStatusBadge with a
    control to set AVAILABLE / ON_BREAK / DELAYED (+minutes input) /
    UNAVAILABLE.
- CurrentPatientCard (largest element): token, patient name, priority

```

badge (hidden if NORMAL), reason for visit if provided, status, live timer since startedAt. Buttons: when CALLED → [Start Consultation] (primary); when IN_CONSULTATION → [Complete Consultation] (primary) + [Skip]. Empty state "No patient called" when none.

- Queue controls row: [Call Next Patient] – biggest button on the screen, disabled while a patient is CALLED/IN_CONSULTATION, while paused, or when no checked-in patient exists (show the reason under the button); [Pause Queue]/[Resume Queue].
- "Today's Queue": next 8 entries in store order. Columns/rows: token, name, PriorityBadge, checked-in indicator, QueueStatusBadge. Not-checked-in rows are muted with "Not arrived".
- Four QueueStatsCards: Patients Served · Currently Waiting · Skipped · Avg Consultation (min, computed from today's completed entries).

/doctor/queue – full list with sections Waiting / Called / Skipped; per-row [Skip] for waiting, [Call] disabled unless first eligible.

/doctor/patients – today's COMPLETED and SKIPPED with times.

/doctor/profile – read-only details, logout.

Doctor must NOT have: cancel, set priority, add walk-in, transfer (Phase 4), manage other doctors, any medical-record fields.

RECEPTION PORTAL /reception/* – optimize for visibility across queues
 Sidebar nav: Dashboard · Queues · Doctors · Appointments · Notifications

/reception/dashboard

- Header: "Coordinator Dashboard – City Care Hospital", today's date.
- Stat cards: Active doctors · Patients waiting · Avg wait · Emergencies today · Not checked in.
- PRIMARY ACTION top-right, always visible: [+ Add Walk-in] → modal: patient name, mobile (optional), doctor (select, grouped by dept), reason (optional), priority (default NORMAL). Dispatches ADD_WALK_IN (checkedIn = true because the patient is physically present) and shows a success toast "Token A-31 created for Mohit – Dr. Ankit Sharma".
- Doctor queue overview: ReceptionDoctorCard per doctor: name, dept, DoctorStatusBadge (DELAYED shows minutes), current token, waiting count, [View Queue]. Card gets amber top border if waiting > 8 ("High load"), red if > 15 ("Critical").
- Live Patient Queue table (all doctors; filter by doctor and status; search by token/name): Token · Patient · Doctor · Priority · Checked-in · Status · Est. wait · Actions.
 Row actions (icon buttons with tooltips): Check In · Set Priority ▾ (NORMAL/PRIORITY/URGENT/EMERGENCY, confirm modal for EMERGENCY) · Transfer ▾ (doctor list) · Skip · Cancel · Notify delay.

/reception/queues – the same table full-page with per-doctor tabs.

/reception/doctors – doctor list; reception can set DELAYED (+minutes) or ON_BREAK; SET_DOCTOR_STATUS triggers delay notifications to that doctor's waiting patients.

/reception/appointments – today's appointments in time order; [Mark arrived] converts the appointment into a WAITING + checkedIn entry with source APPOINTMENT and appointmentTime.

/reception/notifications – sent log + [Broadcast delay notice] form (doctor, minutes, optional message) → pushes to all that doctor's active patients.

Reception must NOT have: start/complete consultation, billing, inventory, payroll, pharmacy, finance, diagnosis, hospital settings.

Cross-portal test checklist – verify every item before finishing

1. Patient joins Dr. Sharma's queue → appears in the doctor's queue and

the reception table immediately.

2. Reception clicks Check In → doctor row loses "Not arrived"; patient token screen shows the green badge.
3. Doctor Call Next → patient screen shows "It's your turn! Room 204"; reception status column = CALLED.
4. Doctor Start → Complete → patient sees "Visit complete"; entry moves to Past visits; doctor stats increment.
5. Reception sets another patient EMERGENCY → they jump to the top of the doctor's queue; everyone else's "ahead" and wait times update.
6. Reception sets Dr. Sharma DELAYED 15 → every waiting patient gets the delay notification; estimated waits increase by 15.
7. Add Walk-in → token appears in doctor queue, already checked in.
8. Doctor Pause → patient sees "Queue paused"; Call Next disabled.
9. Doctor Skip → patient sees red state and [Rejoin]; rejoin returns them to WAITING at the correct position.
10. Demo mode still works.

Deliver

Build passes. docs/STATUS.md updated. Summary as per brief.

PHASE 3 — REGISTRATION & VERIFICATION

PHASE 3 — REGISTRATION & VERIFICATION

Read docs/STATUS.md and docs/DESIGN.md first. Reuse AuthLayout and form primitives from /login. Mock only: no backend, no real OTP, no real license check. No Reception self-registration.

/register — role selection

Title "Create your CareQueue account"; subtext "Choose how you want to use CareQueue". Two RoleSelectionCards:

- User icon · "I'm a Patient" · "Find doctors, join virtual queues, track your turn and manage appointments." · [Register as Patient]
- Stethoscope icon · "I'm a Doctor" · "Manage your patient queue, view today's patients and coordinate consultations." · [Register as Doctor]

Footer: "Reception staff accounts are issued by your hospital."

"Already have an account? Log In"

/register/patient — single page

Title "Create Patient Account"; subtext "Create your account to join queues and manage your appointments."

Personal: Full name* · Mobile* (+91 prefix, 10 digits) · Email* ·

Date of birth* (date picker; no separate age) · Gender (Male / Female / Prefer not to say) · City* · Preferred language (English / Hindi, list driven from data/languages.js)

Security: Password* · Confirm password* — show/hide toggle; min 8 chars incl. 1 number; must match; inline strength hint.

Collapsible "Additional information" (optional): Emergency contact name, Emergency contact number.

Checkboxes: "I agree to the Terms of Service and Privacy Policy."* ·

"I would like to receive queue and appointment notifications." (default on)

[Create Patient Account] · "Already have an account? Log In"

Do NOT ask for Aadhaar, blood group, medical history, symptoms, records.

/verify — mock OTP

Title "Verify Your Account"; "We've sent a verification code to +91 XXXXX XX{last3}". OTPInput: 6 boxes, auto-advance, backspace-back, paste support. [Verify] · [Resend Code] with 30 s countdown. Any 6 digits succeed except "000000" (shows "Invalid code" for demo). On success: add the patient to store users + patients, log in, redirect to /patient/dashboard.

/register/doctor – 4-step Stepper with progress bar

Title "Create Doctor Account"; subtext "Register your professional profile to use CareQueue."

Step 1 Basic Information: Full name* (placeholder "Dr. Full Name") · Mobile* · Professional email*

Step 2 Professional Information: Medical registration number* (helper "This will be used for doctor verification.") · Specialization* (from departments.js + Other) · Years of experience (number, optional) · Profile photo (optional; preview only, no upload)

Step 3 Hospital Association: Hospital / Clinic* – SearchableSelect from hospitals.js, no free text · Department at hospital* (filtered by hospital) · helper "Your hospital association will require approval before your doctor account becomes active."

Step 4 Security & Submit: Password* · Confirm* (same rules) · "I confirm that the professional information provided is accurate."* · "I agree to the Terms of Service and Privacy Policy."* · [Submit for Verification] · "Already registered? Log In"

Back/Next between steps; validate the current step on Next; preserve entered data when going Back; steps clickable only if completed.

/doctor/verification

Clock icon · "Verification Pending" · "Your CareQueue doctor profile has been submitted for verification." Card shows Doctor name, Registration number, Hospital, Department, Status badge "Pending Approval". Text: "You will be able to access your Doctor Dashboard once your professional information and hospital association are approved." [Back to Login]. Store the doctor in users with verified:false. Logging in as an unverified doctor lands here, not on the dashboard. The DEV pill gets an "Approve pending doctors" button that flips verified:true and creates the doctor record (AVAILABLE, avg 10 min, room "TBD").

Validation (lib/validators.js)

Inline errors under fields, no browser alerts. Validate on blur and on submit. Patient: name, valid mobile, valid email, dob, city, password rules, match, terms. Doctor: name, mobile, email, registration number, specialization, department, hospital, password rules, match, both declarations. Keyboard accessible; visible focus rings; labels tied to inputs; error messages announced via aria-live.

Routes to add

/register · /register/patient · /register/doctor · /verify · /doctor/verification. Replace the Phase 1 placeholder. Do not break existing routes.

Components

AuthLayout, RoleSelectionCard, FormInput, SelectInput, SearchableSelect, PasswordInput, CheckboxField, FormSection, Stepper, OTPInput, VerificationStatusCard, FormError – built on components/ui primitives.

Deliver

Build passes. docs/STATUS.md updated. Summary + how to test both flows.

PHASE 4 — HACKATHON DIFFERENTIATORS (WOW LAYER)

PHASE 4 — APPOINTMENTS, QR CHECK-IN, SMART WAIT, ANALYTICS, TRANSFER, PWA
Read docs/STATUS.md first. Everything below extends existing store/pages.

4.1 Appointment booking (patient)

/patient/doctor/:id gets tabs "Join Queue Now" | "Book Appointment".
Booking: date chips (today + 6 days), 15-min slots from todayHours minus booked slots, confirmation card. /patient/visits Upcoming shows [Reschedule] (reopens picker) and [Cancel]. On the appointment day the card shows [Join queue for this appointment] → JOIN_QUEUE with source APPOINTMENT + appointmentTime; the ordering rule then places them ahead of later walk-ins automatically. Show a one-line explainer on the token screen: "Appointment patients are placed by their slot time."

4.2 QR check-in

/reception/qr – large hospital QR (use a QR library to encode `\${origin}/checkin?h=<hospitalId>`), "Print" button, instruction text.
Add "QR Code" to reception nav.
/checkin?h=<hospitalId> – if the logged-in patient has an active WAITING entry at that hospital → CHECK_IN and show a big green confirmation "You're checked in · Token A-24 · 4 ahead · ~30 min" with [Open token].
Not logged in → redirect to /login then back. No active entry → "No active token at this hospital" + link to search.
Patient token screen "Scan QR" modal now shows the encoded URL and a [Simulate scan] button that navigates to it.

4.3 Smart wait estimate (range)

lib/waitTime.js → smartEstimate(entry) returns {low, high}:
base = rolling average of the doctor's last 10 COMPLETED consultations today (fallback avgConsultationMin)
× timeOfDayFactor (0.9 before 11:00, 1.1 between 12:00-14:00, else 1.0)
× sourceFactor (APPOINTMENT 0.95, WALK_IN 1.05)
× patientsAhead, + delayMinutes
low = round(0.85 × total), high = round(1.15 × total)
Token screen shows "Smart estimate ~37-45 min" beneath the simple estimate, with an info tooltip "Learns from today's actual consultation times, time of day and queue mix." Reception table gets a column toggle to show smart range.

4.4 Analytics (reception) /reception/analytics – add to nav

Stat cards: Avg waiting time · Avg consultation time · Skip/no-show rate · Patients served today · Doctor utilization %.
Charts (Recharts allowed, brand colors only): patients per hour today (bar) · waiting per department (bar colored by load) · avg wait last 7 days (line, mock data in data/analytics.js).
Department load table: Department · Waiting · Avg wait · Status (NORMAL < 8, HIGH 8-15, CRITICAL > 15) with colored badges.
Date range selector: Today / This week / This month (mock).

4.5 Transfer patient

TRANSFER UI: reception row action (already present) now works end-to-end; doctor's CurrentPatientCard gets a small "Transfer ▾". Transfer: new token in the target doctor's series, keeps priority and checkedIn, original entry becomes CANCELLED with transferredTo, patient gets notification "You've been transferred to Dr. {name}, Room {room}. New token {t}."

4.6 PWA basics

vite-plugin-pwa: manifest (name CareQueue, short_name CareQueue, theme_color #2563EB, background #FFFFFF, icons 192/512), install prompt banner on patient dashboard. No offline queue logic.

Deliver

Build passes. docs/STATUS.md updated. Write docs/DEMO_SCRIPT.md: a 3-minute judge walkthrough – patient joins → QR check-in → reception inserts EMERGENCY → doctor calls/completes with demo mode → smart estimate → analytics.

PHASE 5 — AWS SERVERLESS BACKEND (POST-MVP)

PHASE 5 – AWS-NATIVE BACKEND (replace mock store internals; keep the UI)
Read docs/STATUS.md first. The QueueStore action names are the API contract. Do not change UI components; replace how the store gets and mutates data. Keep VITE_USE MOCK=true as a working fallback for demos.

Architecture

Frontend: existing React app → AWS Amplify Hosting (or S3 + CloudFront)

Auth: Amazon Cognito User Pool; custom:role (PATIENT|DOCTOR|RECEPTION); groups per role; doctors get custom:verified=false until an admin flips it (simple admin Lambda + CLI script).

API: API Gateway HTTP API (REST) + API Gateway WebSocket API

Compute: AWS Lambda, Node.js 20, one handler per store action, shared queue-engine module (ordering + state-machine table from the brief)

DB: DynamoDB single table "CareQueue", on-demand capacity

Events: EventBridge Scheduler – 10-min grace auto-skip, appointment reminders; SNS SMS optional behind a feature flag

IaC: AWS SAM (template.yaml). Local dev: sam local start-api.

DynamoDB single-table design

PK	SK
HOSPITAL#<id>	META
HOSPITAL#<id>	DOCTOR#<id>
DOCTOR#<id>	DATE#<yyyy-mm-dd>#Q#<sortKey>
sortKey = <priorityRank 0..3><checkedIn 1 0 inverted><joinedAtIso>#<entryId>	
(Query on the DOCTOR#/DATE# prefix returns the queue already in the brief's ordering – no in-memory sort)	
DOCTOR#<id>	DATE#<d>#COUNTER (atomic token counter)
DOCTOR#<id>	DATE#<d>#APPT#<HH:mm>#<id>
PATIENT#<id>	ENTRY#<joinedAtIso>#<entryId> (history)
CONN#<connectionId>	META (attributes: userId, doctorId, hospitalId)

GS1: entryId → item. GS12: hospitalId + DATE#<d> → reception views.

Status changes rewrite the item (delete + put in a transaction) because the sort key encodes priority/checkedIn.

Handlers (mirror store actions; all JWT-authorized; role-checked)

```
POST /queue/join . POST /queue/{id}/cancel . POST /queue/{id}/checkin .  
POST /queue/{id}/rejoin . POST /doctors/{id}/call-next .  
POST /queue/{id}/start . POST /queue/{id}/complete . POST /queue/{id}/skip .  
POST /queue/{id}/priority (RECEPTION) . POST /queue/{id}/transfer .  
POST /walkins (RECEPTION) . PATCH /doctors/{id}/status .  
POST /doctors/{id}/pause . POST /doctors/{id}/resume .  
GET /doctors/{id}/queue . GET /hospitals/{id}/queue . GET /me/entries .  
POST /appointments . PATCH /appointments/{id} . DELETE /appointments/{id} .  
POST /checkin/qrcode {hospitalId}
```

Each mutation: validate transition → write → compute notifications for affected entries → publish {type, doctorId, entries[]} to every CONN# subscribed to that doctor/hospital via the WebSocket management API.
WebSocket routes: \$connect (store conn with JWT claims), \$disconnect, subscribe {doctorId|hospitalId}.

Frontend swap

- store/api.js: fetch wrapper with Cognito token (Amplify Auth).
- Reducer stays; actions become async thunks: optimistic update → API call → reconcile from response or WebSocket event.
- WebSocket client connects after login and dispatches incoming events as the same reducer actions, so no UI component changes.
- Env: VITE_USE MOCK=true keeps the Phase 1 mock behavior.

Deliver

template.yaml, handlers with unit tests for ordering + transitions, README with deploy steps and IAM notes, docs/STATUS.md updated.